APCS 程式設計系列 · U01 · 實作初級 · C++ / Python 對照版

輸入輸出、變數與運算 重點講義

程式的骨架 · 資料型別與溢位 · 整數除法與取餘數 · 小數輸出 · 運算順序

1. 程式的骨架：讀進來、算、印出去

【C++】APCS 實作題全部是「讀標準輸入、寫標準輸出」。C++ 用 **cin** 讀、**cout** 印；cin 會自動跳過空白與換行，所以測資用空白隔開或換行隔開都讀得到。 【Python】APCS 實作題全部是「讀標準輸入、寫標準輸出」。Python 用 **input()** 一次讀一整行（字串），要數字就 **map(int, ...split())** 轉成整數，用 **print()** 印出。

C++ 讀兩個整數，印出和

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;          // 空白、換行隔開都讀得到
    cout << a + b << "\n";  // "\n" 換行，比 endl 快
    return 0;
}
```

Python

```python
a, b = map(int, input().split())  # 一行讀兩個整數
print(a + b)
```

輸入	輸出
3 5	8

Python 讀測資的保險寫法

- input() 一次只讀一行。如果測資把數字拆在好幾行，一行一行讀會出錯。
- 最保險：**import sys** 之後 **data = sys.stdin.read().split()**，把所有數字一次讀成串列再依序取用。
- 資料量大時，這個寫法也比反覆呼叫 input() 快很多。

C++ 加速輸入輸出

- 資料量大（十萬筆以上）時，在 main 開頭加 **ios::sync_with_stdio(false); cin.tie(nullptr);**
- 換行用 **"\n"**，不要用 endl（endl 每次都會清空緩衝區，很慢）。
- 加速之後就不要再混用 printf / scanf 與 cin / cout。

2. 資料型別：數字太大會「溢位」

型別	能存的範圍（約）	什麼時候用
int	$\pm 2.1\times10^9$（$2^{31}-1 = 2147483647$）	一般計數、題目數字 $\le 10^9$
long long	$\pm 9.2\times10^{18}$	相乘、加總可能超過 2×10^9
double	約 15~16 位有效數字	平均、比例等需要小數
Python int	沒有上限（自動變大）	Python 不會溢位，但大數運算比較慢

C++ 相乘前先轉 long long

```cpp
int a = 50000, b = 50000;
long long p = 1LL * a * b;   // 先乘 1LL，整個算式就用 long long 算
cout << p << "\n";
```

Python

```python
a, b = 50000, 50000
print(a * b)   # Python 整數沒有上限
```

輸入	輸出
（不需要輸入）	2500000000

陷阱提醒 long long p = a * b; 為什麼還是錯？

【錯】long long p = a * b; a、b 都是 int，乘法先用 int 算完（已經溢位）才存進 p。

【對】long long p = 1LL * a * b; 或把 a、b 本身宣告成 long long。

- 看到題目範圍有 10^9、$10^5\times10^5$ 這種數字，加總或相乘一律用 long long。

3. 整數除法與取餘數（程式識讀必考）

算式	C / C++ 結果	Python 結果
7 / 2	3（整數相除，小數捨去）	3.5（/ 一定得到小數）
7 // 2	（沒有 // 運算子）	3（整數除法）
7 % 2	1	1
-7 / 2 與 -7 // 2	-3（向 0 取整）	-4（向下取整）
-7 % 2	-1（正負號跟被除數）	1（正負號跟除數）

C++ 實際跑一次看看

```cpp
cout << 7 / 2 << " " << 7 % 2 << " ";
cout << -7 / 2 << " " << -7 % 2 << "\n";
```

Python

```python
print(7 // 2, 7 % 2, -7 // 2, -7 % 2)
```

輸入	輸出
（不需要輸入）	C++： 3 1 -3 -1 Python： 3 1 -4 1

取餘數的三個經典用法

- **拆個位數**：n % 10 是個位數；去掉個位數 C++ 寫 n / 10、Python 寫 n // 10。
- **判斷倍數**：n % k == 0 代表 n 是 k 的倍數（偶數就是 n % 2 == 0）。
- **循環**：今天星期 d（0~6），過 k 天是星期 (d + k) % 7。

陷阱提醒 考場最常錯

- 【C++】[x] 平均寫成 (a + b + c) / 3，三個都是 int，結果被捨去小數。改成 / 3.0。【Python】[x] 該用整數除法卻寫 /，印出 3.0 而不是 3。整數答案一律用 //。

【錯】負數的除法與取餘數，C 和 Python 結果不同。考程式識讀時看清楚是哪一種題本。

4. 小數輸出：指定印到小數點後幾位

C++ 三科平均，印到小數第 2 位（四捨五入）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    double avg = (a + b + c) / 3.0;          // 3.0 讓除法變成小數除法
    cout << fixed << setprecision(2) << avg << "\n";
    // 也可以寫 printf("%.2f\n", avg);
}
```

Python

```python
a, b, c = map(int, input().split())
avg = (a + b + c) / 3
print(f"{avg:.2f}")   # 或 print("%.2f" % avg)
```

輸入	輸出
80 75 92	82.33

5. 運算順序與遞增

- 先算 * / %（同一層由左到右），再算 + -；不確定就加括號。
- 17 % 5 * 3 + 2：先算 17 % 5 = 2，再 2 * 3 = 6，最後 + 2 = 8。
- x += 3 等於 x = x + 3。C++ 的 x++ 是「先用再加」、++x 是「先加再用」；Python 沒有 ++。

C++ 先用再加

```cpp
int x = 5;
int y = x++ * 2;     // 先用 x=5 算 y=10，然後 x 變 6
cout << x << " " << y << "\n";
```

Python

```python
x = 5
y = x * 2    # Python 要分兩行寫
x += 1
print(x, y)
```

輸入	輸出
（不需要輸入）	`6 10`

讀完這份，你應該要會

- ☐ 會寫出讀兩個整數、印出和的完整程式（兩種語言）
- ☐ 知道 int 的上限約 2.1×10^9，看到大數會改用 long long（Python 不用擔心）
- ☐ 7 / 2、7 % 2、-7 / 2 在 C 和 Python 分別是多少，閉著眼睛都答得出來
- ☐ 會用 % 10 和除以 10 拆出個位數、十位數
- ☐ 會印到小數點後 2 位

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 jenfoo2021